AccessAI

Bangladesh's AI Opportunity Intelligence Platform

Product Requirements Document (PRD)

Version: 3.0

Document Type: Master Product Specification

Project Status: Production-Oriented MVP

Prepared For: Bangladesh ICT & Innovation Awards 2026

---

1. Introduction

1.1 Product Name

AccessAI

1.2 Product Tagline

One AI Assistant for Every Citizen.

1.3 Product Overview

AccessAI is an AI-powered Opportunity Intelligence Platform that enables citizens to discover, understand, qualify for, and access government services, NGO programs, scholarships, healthcare facilities, financial assistance, legal support, agricultural services, and public opportunities through natural language conversations.

Instead of requiring citizens to search hundreds of government websites and understand complex ministry structures, AccessAI understands the citizen's situation and automatically identifies the opportunities they are eligible for.

Unlike traditional search engines or government portals, AccessAI acts as an intelligent decision-support system.

It combines:

- Large Language Models
- Retrieval-Augmented Generation (RAG)
- Rule-Based Eligibility Evaluation
- Semantic Search
- Knowledge Graphs
- Recommendation Systems
- Explainable AI

to generate trusted, explainable, and actionable recommendations.

The objective is not merely answering questions.

The objective is helping citizens make correct decisions.

1.4 Vision

To build Bangladesh's most trusted AI-powered gateway for discovering and accessing public opportunities, ensuring that every citizen—regardless of education, location, language, or digital literacy—can confidently access the services designed to improve their lives.

1.5 Mission

AccessAI exists to eliminate information inequality by transforming fragmented public information into personalized, explainable, and actionable guidance.

Instead of asking citizens to understand government systems, AccessAI enables government systems to understand citizens.

1.6 Product Philosophy

Government services are currently designed around institutions.

Citizens think about life events.

This mismatch creates unnecessary complexity.

AccessAI changes the interaction model.

Instead of asking:

> Which government department are you looking for?

The platform asks:

> Tell me what happened in your life.

This small change fundamentally transforms public service accessibility.

---

2. Problem Statement

Bangladesh has made significant investments in digital governance over the past decade.

Thousands of services are available online.

However, accessibility remains a major challenge.

Information is fragmented across:

- Ministry websites
- Department portals
- Government circulars
- NGO websites
- Scholarship portals
- Hospital websites
- PDF documents
- Facebook announcements
- Union Digital Centers

Every organization maintains its own website.

Every website follows different structures.

Every program has different eligibility requirements.

Citizens must manually search dozens of websites to determine whether they qualify for assistance.

Most people never complete this process.

As a result,

valuable opportunities remain undiscovered.

Government services remain underutilized.

Citizens lose access to benefits they are legally entitled to receive.

---

3. Existing Problems

Information Fragmentation

There is no unified platform that aggregates verified opportunities across government, NGOs, healthcare providers, educational institutions, and financial organizations.

No Personalized Guidance

Government portals display information.

They do not understand users.

Every visitor receives identical information regardless of their background.

Complex Eligibility Rules

Many government services involve multiple eligibility conditions.

Examples include:

- Age
- Occupation
- Income

- Gender
- District
- Educational Qualification
- Disability Status
- Land Ownership
- Marital Status

Citizens often cannot determine whether they qualify.

Technical Language

Government circulars frequently use legal and administrative language that ordinary citizens struggle to understand.

Poor Discoverability

Citizens generally discover opportunities through:

- Friends
- Social Media
- Local Offices
- Word of Mouth

instead of official channels.

This significantly reduces awareness.

Duplicate Searching

Users repeatedly visit multiple websites searching for similar information.

There is no unified discovery experience.

Lack of Trust

Modern AI systems often generate hallucinated answers.

Citizens require trustworthy recommendations supported by official evidence.

---

4. Product Goal

The primary objective of AccessAI is to become Bangladesh's intelligent public opportunity discovery platform.

The platform should:

- Understand natural language.
- Understand life situations.
- Evaluate eligibility.
- Discover opportunities.
- Explain recommendations.
- Generate personalized action plans.
- Increase public service utilization.
- Reduce information inequality.

---

5. Core Concept

AccessAI is NOT a chatbot.

AccessAI is NOT a search engine.

AccessAI is NOT another government portal.

AccessAI is an AI Decision Support Platform.

Its responsibility is to answer one question:

> Based on my current situation, what should I do next?

---

6. Product Positioning

Traditional Government Portal

↓

Shows information.

Google

↓

Shows links.

ChatGPT

↓

Generates text.

AccessAI

↓

Generates decisions.

---

7. Product Principles

Every feature inside AccessAI must satisfy these principles.

Principle 1

Citizen First

Every workflow starts with the citizen.

Never with ministries.

Principle 2

Explain Everything

Every recommendation must explain

WHY.

Principle 3

Evidence Required

Every recommendation must reference

verified sources.

No unsupported claims.

Principle 4

AI Assists

AI should explain.

Rules should decide.

Business logic must never depend entirely on an LLM.

Principle 5

Simple Language

Every explanation should be understandable by someone with basic education.

Avoid bureaucratic terminology.

Principle 6

Actionable Guidance

Never end with information.

Always end with next steps.

---

8. Target Users

Primary users include:

- Students
- Farmers
- Women
- Elderly Citizens
- Persons with Disabilities
- Entrepreneurs
- Small Businesses
- Job Seekers
- Low Income Families
- Disaster Affected Communities
- Healthcare Seekers
- Researchers

---

9. Primary User Goals

Users visit AccessAI to:

- Discover opportunities
- Determine eligibility
- Find scholarships
- Locate nearby offices
- Learn required documents
- Understand government processes
- Find NGO support
- Generate action plans
- Track deadlines
- Receive reminders

---

10. Product Scope

In Scope

Government Programs

NGO Programs

Scholarships

Healthcare Services

Agricultural Programs

SME Support

Legal Aid

Financial Inclusion

Nearby Service Centers

AI Chat

Voice Assistant

Eligibility Engine

Recommendation Engine

Document Checklist

Opportunity Timeline

Notifications

Multilingual Support

Admin Dashboard

Analytics Dashboard

Knowledge Base Management

Out of Scope (MVP)

Government payment processing

National ID verification

Official application submission

Government workflow automation

Digital signatures

Benefit approval

Government decision making

Bank transaction processing

Identity authentication through government APIs

These features may be added in future versions through official integrations.

---

11. Success Criteria

The MVP will be considered successful if it can:

- Answer public service queries accurately.
- Generate personalized recommendations.
- Explain eligibility with evidence.
- Produce actionable guidance.
- Retrieve information within three seconds.
- Maintain response accuracy above predefined evaluation thresholds.
- Support Bangla and English interactions.
- Operate with minimal hallucination through retrieval-based responses.

---

12. Product Vision Statement

AccessAI is an AI-powered Opportunity Intelligence Platform that understands a citizen's life situation, determines eligibility across government and NGO services, explains every

recommendation using trusted evidence, and generates personalized action plans that enable citizens to confidently access opportunities designed to improve their lives.

AccessAI

Part 2 — Complete Product Specification

---

13. Product Overview

AccessAI is an AI-native decision support platform designed to bridge the gap between citizens and public opportunities. The platform does not replace existing government portals; instead, it acts as an intelligent layer above them.

Instead of expecting users to know which ministry provides a service, AccessAI understands the user's life situation, identifies relevant opportunities, evaluates eligibility using verified rules, retrieves official information, and generates personalized guidance.

The platform is built around one central philosophy:

> **People describe their lives. AI discovers opportunities.**

---

14. Core Product Workflow

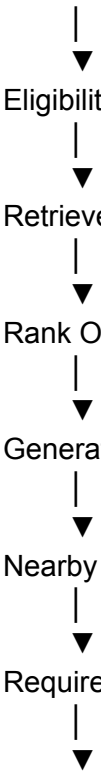
Every interaction inside AccessAI follows the same workflow.

User

Describe Situation

AI Understands Context

Extract User Information



Every feature in the platform extends this workflow.

---

15. Core Features

Feature 1 — AI Conversation Engine

Objective

Allow users to communicate naturally using Bangla or English without requiring knowledge of government structures.

Description

The AI Conversation Engine is the primary interface between citizens and the platform.

Instead of navigating menus, users simply describe their current situation.

Example:

> "I recently graduated and I need financial support."

The system should continue asking follow-up questions until sufficient information is collected.

Responsibilities

- Understand natural language
- Maintain conversation history
- Ask follow-up questions
- Detect missing information
- Guide users through complex scenarios
- Hand over structured data to downstream AI services

Inputs

- Text
- Voice
- Uploaded Documents (Future)
- Uploaded Images (Future)

Outputs

- Structured user profile
- Conversation summary
- Intent
- Extracted entities

Acceptance Criteria

- Supports Bangla and English
- Maintains conversation context
- Handles incomplete sentences
- Requests clarification when necessary
- Responds within 3 seconds

Feature 2 — Life Event Intelligence

Objective

Convert human situations into machine-understandable events.

Description

Users rarely know which government service they require.

Instead, they know what happened in their lives.

Examples include:

- Lost my job
- Want to study abroad
- Need cancer treatment
- Recently became a widow
- Want to start a business
- House damaged by flood
- Looking for employment
- Need disability support

The platform should recognize these life events and map them to multiple public opportunities.

Example

Input

My husband passed away.

Detected Event

Widow

Triggered Services

Widow Allowance

VGD

Healthcare

Legal Aid

Livelihood Support

Educational Support

Mental Health Services

Acceptance Criteria

One life event may activate multiple services simultaneously.

Feature 3 — Intelligent Eligibility Engine

Objective

Determine eligibility using verified government rules.

Description

Eligibility should never depend solely on an LLM.

Instead, deterministic business rules must evaluate every program.

The AI's responsibility is to explain—not decide.

Inputs

Age

Gender

District

Income

Occupation

Education

Disability

Marital Status

Household Information

Land Ownership

Student Status

Business Status

Outputs

Eligible

Partially Eligible

Not Eligible

Unknown

Acceptance Criteria

Every decision must include reasoning.

Example:

Eligible

Reason

Age above 65

Income below threshold

Bangladeshi citizen

Feature 4 — Opportunity Discovery Engine

Objective

Identify every opportunity relevant to a user.

Description

Instead of returning a single answer, AccessAI searches across all verified knowledge sources.

These include

Government

NGOs

Scholarships

Healthcare

Agriculture

SMEs

Training

Financial Programs

Legal Aid

Community Services

Every recommendation receives a relevance score.

Feature 5 — Opportunity Graph ⭐

This is one of the platform's most important differentiators.

Instead of showing isolated recommendations, AccessAI builds a connected opportunity graph.

Example

Widow

│

├── Widow Allowance

├── VGD

├── Healthcare

├── Educational Support

├── BRAC

├── Livelihood Training

├── Free Legal Aid

└── Microfinance

Users understand the entire ecosystem instead of isolated programs.

Feature 6 — Explainable AI

Every recommendation must answer

Why?

Example

You qualify because

Your age satisfies the requirement.

Your income falls below the threshold.

Your district is covered.

Your occupation is eligible.

No black-box recommendations.

Feature 7 — Personalized Action Planner

Recommendations alone are insufficient.

Every recommendation should become a task list.

Example

Today

Collect National ID

Tomorrow

Collect Income Certificate

Wednesday

Visit Union Digital Center

Thursday

Submit Application

Saturday

Track Application Status

The user should always know what to do next.

Feature 8 — Document Intelligence

For every program the system should automatically generate

Required Documents

Optional Documents

Issuing Authority

Estimated Processing Time

Renewal Schedule

Common Mistakes

Preparation Tips

Example

Widow Allowance

Required

National ID

Death Certificate

Passport Photo

Income Certificate

Family Certificate

Estimated Processing

7–14 Days

Feature 9 — Scholarship Intelligence

The platform should maintain a centralized scholarship database.

Students provide

CGPA

Department

University

Nationality

Research Interests

Preferred Country

Financial Background

Language Scores

The system automatically

Ranks scholarships

Calculates eligibility

Explains requirements

Shows deadlines

Creates application timeline

Suggests preparation strategy

Feature 10 — Healthcare Navigator

Users should be able to discover

Government Hospitals

Community Clinics

Specialized Hospitals

Maternal Health

Cancer Support

Mental Health

Emergency Services

Disability Centers

Nearby Pharmacies

Healthcare recommendations should consider

Location

Age

Medical Need

Government Coverage

Feature 11 — NGO Discovery

Many NGOs provide assistance unavailable through government programs.

AccessAI should recommend

Educational NGOs

Healthcare NGOs

Women's Organizations

Livelihood Programs

Disaster Relief

Legal Aid

Youth Development

Climate Adaptation

Every NGO should include

Description

Eligibility

Coverage Area

Contact Information

Programs

Office Locations

Feature 12 — Nearby Service Finder

For every recommendation

Locate

Nearby Union Office

Government Office

Hospital

NGO

Training Center

Bank

Legal Aid Office

Agricultural Extension Office

Each location includes

Distance

Office Hours

Navigation

Contact

Feature 13 — Opportunity Timeline ⭐

Every user receives a personalized timeline.

Timeline includes

Upcoming Deadlines

Document Expiry

Scholarship Windows

Renewals

Training Sessions

Government Announcements

Application Progress

Reminders

This transforms AccessAI from reactive AI into proactive AI.

Feature 14 — Notification Engine

Notify users when

Applications open

Deadlines approach

Programs change

New opportunities appear

Documents expire

Recommendations improve

Notifications should support

Push

Email

SMS (Future)

WhatsApp (Future)

Feature 15 — Voice Assistant

Users may interact completely through speech.

Capabilities

Speech to Text

Voice Commands

Text to Speech

Conversation

Navigation

Accessibility Mode

Priority users

Elderly

Visually Impaired

Low Literacy Users

Feature 16 — Multilingual Support

Initial Languages

Bangla

English

Future

Regional Dialects

Automatic Language Detection

Feature 17 — User Profile

The system should maintain a dynamic user profile.

Stored Information

Basic Information

Education

Occupation

Household

Income

Location

Health Preferences

Conversation History

Saved Opportunities

Application History

Notification Preferences

The profile improves recommendations over time.

Feature 18 — Opportunity Tracker

Instead of only recommending programs,

the platform should help users track them.

Each opportunity contains

Status

Interested

Preparing

Documents Ready

Applied

Under Review

Approved

Rejected

Completed

Users can monitor progress from a single dashboard.

Feature 19 — Trust Dashboard

Every recommendation displays

Confidence Score

Last Updated

Official Source

Retrieved Documents

Eligibility Explanation

AI Reasoning Summary

Data Freshness

Verification Status

Transparency is a core design principle.

Feature 20 — Admin Portal

Administrators can

Manage Programs

Manage Organizations

Upload PDFs

Upload Circulars

Update Rules

Manage Locations

Review AI Logs

Review Conversations

Manage Embeddings

Manage Notifications

View Analytics

Approve Data Changes

Rebuild Search Index

No developer intervention should be required for routine content updates.

---

16. User Journey

Step 1

User opens AccessAI.

↓

Step 2

User selects

Text

or

Voice.

↓

Step 3

User explains their situation.

↓

Step 4

AI asks follow-up questions.

↓

Step 5

Eligibility Engine evaluates programs.

↓

Step 6

Knowledge Engine retrieves official information.

↓

Step 7

Recommendation Engine ranks opportunities.

↓

Step 8

User receives

- Opportunities
- Reasons
- Required Documents
- Nearby Offices
- Timeline
- Action Plan

↓

Step 9

User saves recommendations.

↓

Step 10

Platform continues monitoring future opportunities automatically.

---

17. Product Success Metrics

The platform should continuously measure:

- Recommendation Accuracy
- Eligibility Accuracy
- Hallucination Rate
- Citation Coverage
- Average Response Time
- User Satisfaction
- Opportunity Discovery Rate
- Successful Application Rate
- Daily Active Users
- Monthly Active Users
- Retention Rate
- Saved Opportunities
- Completed Action Plans
- Notification Engagement
- Search-to-Application Conversion Rate

AccessAI

Part 3 — AI System Architecture & Intelligence Layer

> **Purpose:** This section defines the complete AI architecture required to implement AccessAI. The system is designed as a modular AI decision-support platform rather than a single chatbot. All AI components must work together through clearly defined pipelines, ensuring scalability, explainability, and high reliability.

---

18. AI Design Philosophy

The platform must **not** depend on a single Large Language Model for all reasoning.

Instead, the AI system should combine deterministic business logic, structured knowledge, semantic retrieval, and LLM reasoning.

The guiding principle is:

> **Rules determine facts. AI explains facts.**

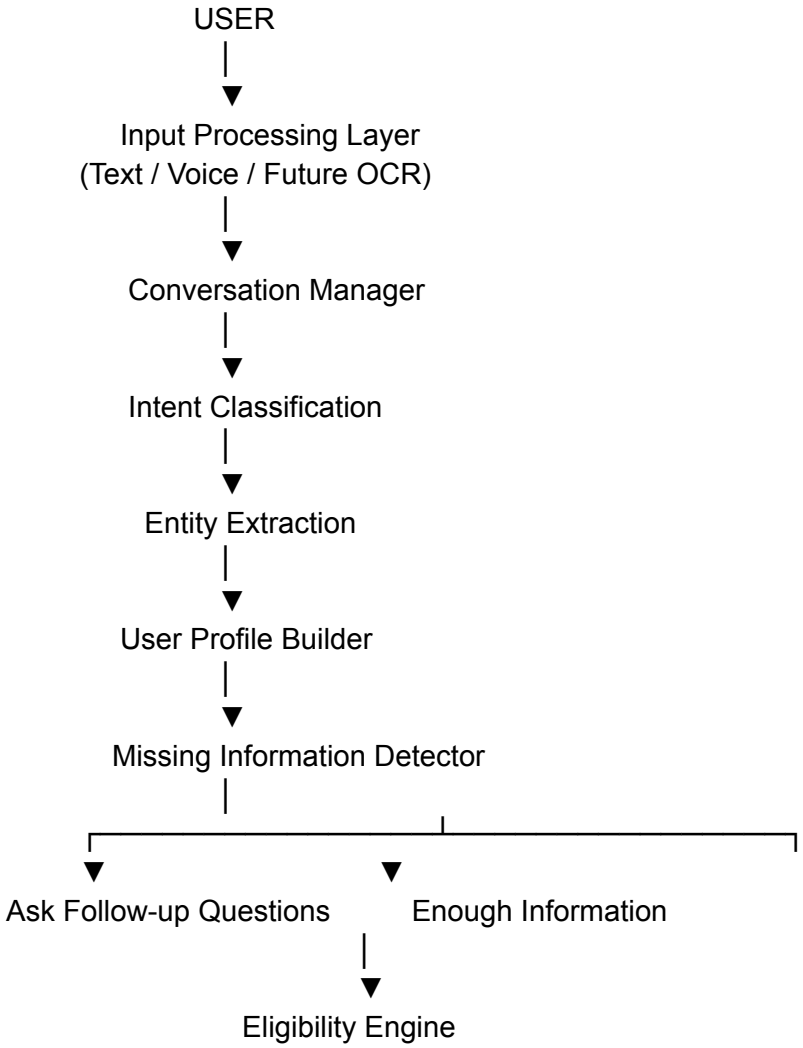
For example:

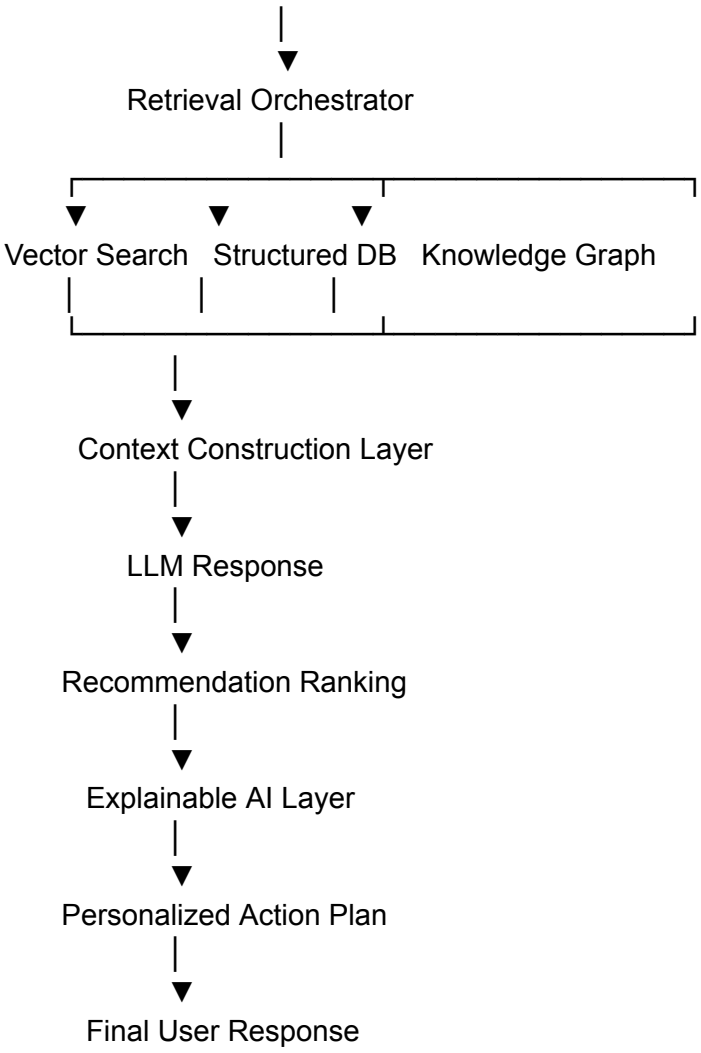
- The Eligibility Engine determines whether a user qualifies.
- The LLM explains why the user qualifies.
- The Recommendation Engine ranks opportunities.
- The LLM generates a human-friendly explanation.

This architecture minimizes hallucination while improving trust and transparency.

---

19. High-Level AI Architecture





---

20. AI Modules

The AI system is divided into independent services.

Each module has a single responsibility.

Module 1 — Conversation Manager

Purpose

Maintain the complete conversation.

Responsibilities

- Store conversation history
- Preserve context
- Handle follow-up questions
- Resolve references

Example

User:

> I lost my job.

Later:

> Can I still apply?

The AI understands that "apply" refers to the previously discussed employment assistance.

Module 2 — Intent Classifier

Purpose

Determine what the user actually wants.

Possible intents include:

- Find Opportunities
- Check Eligibility
- Scholarship Search
- Healthcare Search
- NGO Assistance
- Agriculture Support
- SME Support
- Ask Question
- Document Requirements
- Nearby Services
- Timeline Request
- Follow-up Question
- General Information

A single query may contain multiple intents.

Example

I recently graduated and want to study abroad.

Detected Intents:

- Scholarship Search
- Study Abroad
- Financial Assistance

Module 3 — Entity Extraction

Purpose

Extract structured information from natural language.

Supported entities include:

Personal

- Name
- Age
- Gender

Location

- Division
- District
- Upazila

Financial

- Income
- Employment
- Business

Education

- University
- Department
- CGPA
- Degree

Household

- Marital Status
- Dependents
- Family Size

Health

- Disability
- Medical Condition (only when relevant)
- Pregnancy

Agriculture

- Farm Size
- Crops
- Livestock

Business

- Company Type
- Registration
- Employees

Entities are stored in a structured user profile.

---

21. Dynamic User Profile

The profile grows over time.

Never ask users for information already known.

Example

```
{
   "age":22,
   "district":"Dhaka",
   "occupation":"Student",
   "cgpa":3.82,
   "income":0,
   "interests":[
      "Scholarships",
      "AI"
   ]
```

}

Each conversation updates this profile.

---

22. Missing Information Detector

Before performing eligibility evaluation, determine whether required information exists.

Example

Scholarship eligibility requires:

- CGPA
- Degree
- Nationality

If CGPA is missing,

the AI asks:

> What is your current CGPA?

instead of guessing.

---

23. Life Event Detection Engine

Life events are one of the platform's core innovations.

The engine converts free-text into standardized events.

Example mapping:

User Statement	Detected Event
I lost my job	Job Loss
My husband passed away	Widowhood

I want to study abroad	Higher Education
I need cancer treatment	Serious Medical Need
I want to start a business	Entrepreneurship
Flood destroyed our house	Disaster Recovery

A user may have multiple simultaneous life events.

---

24. Eligibility Engine

This module **must not use an LLM**.

Instead, use deterministic rules.

Example rule:

```
Program:
  Widow Allowance

Rules:
  Gender == Female
  MaritalStatus == Widow
  Income < Threshold
  Citizenship == Bangladeshi
```

Evaluation result:

```
{
   "eligible":true,
   "confidence":100,
   "matched_rules":4,
   "failed_rules":0
}
```

The LLM receives this result and explains it.

---

25. Retrieval Orchestrator

Purpose

Gather relevant information before generating a response.

Sources:

- Vector Database
- PostgreSQL
- Knowledge Graph
- Static Metadata
- Organization Database

The orchestrator combines these sources into a single context package.

---

26. Hybrid Retrieval Strategy

The platform should never rely solely on vector search.

Use hybrid retrieval:

Semantic Search
+
Keyword Search
+
Metadata Filtering
+
Structured Queries

Example:

Query

> Scholarships for female engineering students

Retrieval:

- Semantic similarity
- Category = Scholarship
- Gender = Female
- Engineering
- Deadline not expired

Hybrid retrieval improves precision.

---

27. Knowledge Graph

Relationships between entities should be stored explicitly.

Example:

Student

│

├── Scholarship

├── Internship

├── Training

├── Research Grant

└── Exchange Program

Another example:

Farmer

│

├── Subsidy

├── Crop Insurance

├── Agricultural Loan

├── Fertilizer Support

└── Training

Knowledge graphs allow indirect opportunity discovery.

---

28. Context Builder

Before calling the LLM, create a structured context.

Context includes:

- User Profile
- Conversation Summary
- Retrieved Documents
- Eligibility Results
- Nearby Services
- Opportunity Rankings
- Relevant Policies

Only relevant information should be included.

Avoid prompt overload.

---

29. Prompt Construction

The prompt should follow this structure:

System Prompt

↓

User Profile

↓

Conversation Summary

↓

Eligibility Results

↓

Retrieved Context

↓

Relevant Programs

↓

Instructions

↓

User Question

The LLM should never receive raw database dumps.

---

30. Explainable AI Layer

Every recommendation must explain:

- Why recommended
- Why not recommended
- Required documents
- Responsible authority
- Confidence score
- Source of information

Users should understand every decision.

---

31. Recommendation Engine

Recommendations should be ranked using weighted scoring.

Example:

Factor	Weight
Eligibility Match	40%
User Preference	15%
Location	15%
Deadline	10%
Popularity	10%
Similar User Success	10%

The highest-scoring opportunities appear first.

---

32. Confidence Scoring

Every AI response receives a confidence score.

Factors:

- Retrieval quality
- Rule completeness
- Number of supporting sources
- Data freshness
- Metadata quality

Example:

Scholarship Recommendation

Confidence

96%

Reason

5 official documents found

Eligibility verified

Deadline valid

Updated last week

---

33. Hallucination Prevention

The platform should minimize hallucinations using multiple safeguards:

1. Never answer without retrieved evidence.
2. Eligibility is determined only by the rule engine.
3. If no reliable information is found, clearly state that instead of guessing.
4. Cite the official organization for every recommendation.
5. Show the last update date of retrieved information.

The LLM must never invent programs, eligibility rules, deadlines, or organizations.

---

34. Learning & Feedback Loop

The system continuously improves through user feedback.

After each interaction, users can:

- Mark the response as helpful or not.
- Report incorrect information.
- Suggest missing opportunities.
- Rate recommendation quality.

Feedback is stored for analytics and future improvements, but it must **not** automatically change eligibility rules. Administrative review is required before updating verified knowledge.

---

35. AI Evaluation Metrics

To monitor model performance, track:

Metric	Description
Intent Classification Accuracy	Correctly identified user intent
Entity Extraction Accuracy	Correctly extracted structured data
Eligibility Accuracy	Match with official rules
Recommendation Precision	Relevant opportunities returned
Recommendation Recall	Percentage of available opportunities discovered
Hallucination Rate	Unsupported AI-generated claims
Citation Coverage	Responses backed by official sources
Average Response Time	End-to-end latency
User Satisfaction	User ratings after conversations

---

36. AI Principles

Every AI component must follow these principles:

- AI **assists**, it does not make legal or administrative decisions.
- Deterministic business rules take precedence over LLM reasoning.
- Every recommendation must be explainable.
- Every recommendation must be traceable to verified data.
- Conversations should be simple, conversational, and accessible.
- Transparency is more important than sounding intelligent.
- When uncertainty exists, the system should admit uncertainty rather than fabricate information.

This completes **Part 3**, which defines the intelligence layer of AccessAI. The next section (**Part 4**) will specify the **backend engineering architecture**, including the complete database schema, API design, authentication model, storage strategy, background jobs, deployment architecture, caching, monitoring, logging, and scalability plan. This section will serve as the implementation blueprint for the engineering team.

Good. This is the most important engineering section. If an AI editor (Cursor/Codex/Claude Code) receives only this section, it should be able to scaffold almost the entire backend.

AccessAI

Part 4 — Backend Architecture & System Design

---

37. Technology Stack

The platform should be designed as a modern, scalable, cloud-native web application following a modular monolith architecture initially, with the ability to migrate individual modules into microservices as the platform grows.

Frontend

- Next.js (App Router)
- TypeScript
- Tailwind CSS
- shadcn/ui
- React Query (TanStack Query)
- Zustand
- Framer Motion
- React Hook Form
- Zod

Backend

- NestJS
- TypeScript

Reason:

- Modular Architecture
- Dependency Injection
- Enterprise-ready
- Built-in Validation
- Easy Testing
- Excellent Swagger Support

Database

PostgreSQL

Extensions

- pgvector
- uuid-ossp

Cache

Redis

Uses

- Session cache
- Rate limiting
- AI response cache
- Frequently used opportunities
- Background queues

Vector Database

Use PostgreSQL with pgvector.

No Pinecone or Weaviate is required for MVP.

Benefits

- Simpler deployment
- Lower cost

- Single database
- Easier backup
- Better consistency

Search

Hybrid Search

- PostgreSQL Full Text Search
- pgvector Similarity Search

Object Storage

Cloudflare R2

or

AWS S3

Stores

PDFs

Circulars

Images

Embeddings Metadata

User Uploads

Queue

BullMQ

Background jobs include

Embedding generation

PDF parsing

Notification sending

Daily synchronization

Data crawling

Analytics

Authentication

JWT

Refresh Tokens

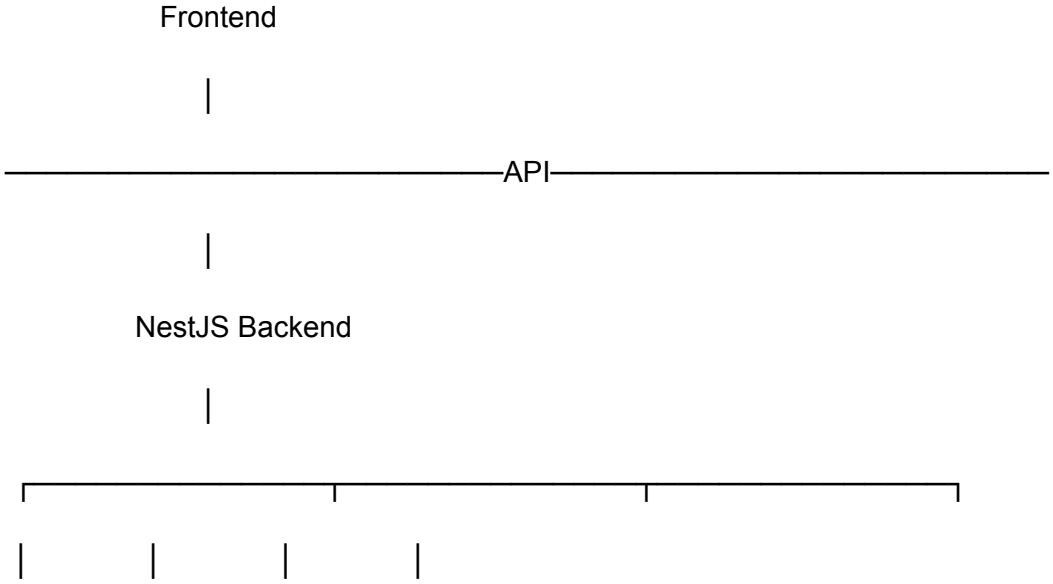
Google Login

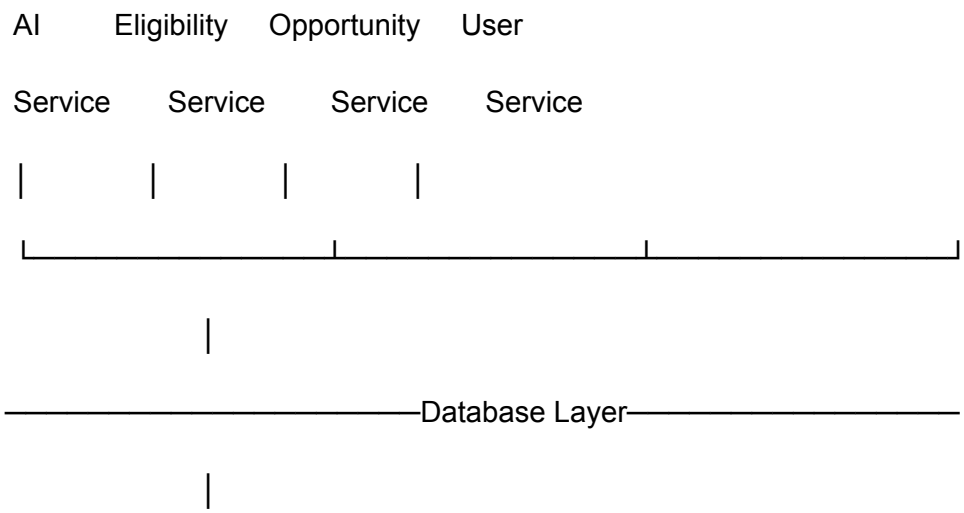
Future

National ID Integration

---

38. High-Level Backend Architecture





---

39. Backend Modules

Each module must be isolated.

Authentication Module

Responsibilities

User Registration

Login

JWT

Refresh Token

Password Reset

OAuth

Role Management

User Module

Stores

Profile

Preferences

Saved Opportunities

Application History

Conversation Summary

Notification Settings

AI Module

Responsible for

Prompt Construction

Conversation

Context Building

Response Generation

Memory

Follow-up Questions

Opportunity Module

Stores

Programs

Scholarships

NGOs

Hospitals

Legal Aid

Agriculture

Financial Programs

Business Support

Eligibility Module

Evaluates

Business Rules

Program Requirements

Matching Logic

Scoring

Search Module

Handles

Semantic Search

Keyword Search

Hybrid Ranking

Filtering

Notification Module

Push

Email

Future SMS

Future WhatsApp

Analytics Module

Tracks

Usage

Recommendation Quality

User Growth

Errors

Performance

Admin Module

Program Management

Knowledge Base

Documents

Users

Reports

Moderation

---

40. Database Schema

The database should use UUIDs for all primary keys.

User

id

name

email

phone

password_hash

role

language

district

created_at

updated_at

UserProfile

id

user_id

date_of_birth

gender

occupation

income

marital_status

education

cgpa

university

department

disability

household_size

Organizations

id

name

type

description

website

contact

address

district

verified

Types

Government

NGO

Hospital

University

Bank

Training Center

Opportunities

id

organization_id

title

category

description

requirements

benefits

deadline

status

created_at

updated_at

EligibilityRules

id

opportunity_id

rule_json

priority

version

Store rules as JSON.

OpportunityCategories

Scholarship

Healthcare

Agriculture

Business

Legal Aid

Employment

Financial

Social Welfare

Training

Disaster

Research

Documents

id

opportunity_id

file_url

type

version

source

embedding_status

Embeddings

id

document_id

chunk_index

embedding

metadata

Uses pgvector.

Conversations

id

user_id

summary

started_at

ended_at

Messages

id

conversation_id

role

content

tokens

created_at

Saved Opportunities

user_id

opportunity_id

status

saved_at

Status

Interested

Preparing

Applied

Rejected

Completed

Notifications

id

user_id

title

body

type

read

scheduled_at

---

41. Database Relationships

User

│

├── User Profile

├── Conversations

├── Saved Opportunities

├── Notifications

└── Settings

Opportunity

│

├── Organization

├── Documents

├── Eligibility Rules

├── Locations

└── Categories

---

42. REST API Design

Authentication

POST /auth/register

POST /auth/login

POST /auth/logout

POST /auth/refresh

GET /auth/me

User

GET /users/me

PATCH /users/me

GET /users/profile

PATCH /users/profile

Conversation

POST /chat

GET /chat/history

GET /chat/:id

DELETE /chat/:id

Opportunity

GET /opportunities

GET /opportunities/:id

POST /opportunities/search

GET /opportunities/recommended

Eligibility

POST /eligibility/check

POST /eligibility/explain

Organizations

GET /organizations

GET /organizations/:id

Documents

GET /documents

POST /documents/upload

POST /documents/process

Notifications

GET /notifications

PATCH /notifications/read

Admin

POST /admin/program

PATCH /admin/program

DELETE /admin/program

POST /admin/document

POST /admin/reindex

POST /admin/rebuild-embeddings

---

43. Authentication Flow

User

↓

Login

↓

JWT

↓

Refresh Token

↓

API

↓

Protected Route

↓

Authorized Resource

Roles

Guest

Citizen

Moderator

Administrator

Super Admin

---

44. Caching Strategy

Redis should cache

Frequently requested opportunities

Popular scholarships

Nearby organizations

Frequently asked questions

Conversation summaries

User preferences

Embedding search results

---

45. Background Jobs

BullMQ workers

Job Types

Daily document synchronization

Embedding generation

PDF parsing

Notification scheduling

Expired opportunity cleanup

Recommendation recalculation

Analytics aggregation

Dead link detection

---

46. Logging

Use Pino Logger.

Log

API requests

Errors

Authentication

AI latency

Embedding latency

Search latency

Recommendation latency

---

47. Monitoring

Use OpenTelemetry.

Track

CPU

Memory

Latency

Queue Length

Database Performance

Embedding Generation Time

Search Performance

Cache Hit Rate

API Errors

---

48. Security

Password

Argon2 hashing

Authentication

JWT

HTTPS

Always enabled

Database

Parameterized queries only

Validation

Zod

NestJS ValidationPipe

Headers

Helmet

CORS

Configured

Rate Limiting

Redis

Uploads

Virus scan

Allowed MIME types

Maximum file size

Prompt Injection Protection

Sanitize user prompts before constructing AI context.

Never allow prompts to override system instructions or expose internal implementation details.

---

49. Scalability Strategy

The MVP starts as a **modular monolith**.

Future services that can be extracted independently:

- AI Service
- Search Service
- Notification Service
- Analytics Service
- Document Processing Service

This avoids premature microservice complexity while allowing future scaling.

---

50. Error Handling

The API should return consistent error responses.

Example:

```
{
  "success": false,
  "error": {
    "code": "ELIGIBILITY_RULE_NOT_FOUND",
    "message": "Eligibility rules are unavailable for this opportunity."
  },
  "timestamp": "2026-08-15T10:30:00Z",
  "requestId": "uuid"
}
```

All endpoints should include structured logging, request IDs, and meaningful HTTP status codes.

---

51. Engineering Standards

- Follow Clean Architecture principles.
- Keep business logic inside services, not controllers.
- Use repositories only for data access.
- Never embed AI prompts directly in controllers.
- Store prompts as versioned templates.
- Write unit tests for eligibility rules.
- Validate all API inputs.
- Use database migrations for schema changes.
- Version public APIs (`/api/v1/...`) from the start.
- Generate OpenAPI (Swagger) documentation automatically.

This completes **Part 4**. It defines the backend implementation blueprint, database, APIs, infrastructure, security, and engineering conventions. The next section (**Part 5**) will specify the complete frontend, UX, design system, navigation, every page, every component, and every user interaction in implementation-level detail.

Excellent. This section defines **everything the user sees and interacts with**. It is detailed enough for an AI coding assistant to generate the frontend architecture, routing, components, and state management.

AccessAI

Part 5 — Frontend Architecture, UX & Design System

---

52. Frontend Philosophy

The frontend must prioritize **clarity, accessibility, and trust** over visual complexity.

The UI should feel more like a **trusted public service assistant** than a chatbot.

Core principles:

- Mobile-first responsive design
- Minimal cognitive load
- Accessibility (WCAG AA)
- Fast interactions
- Conversational guidance
- Explainable recommendations
- Consistent design language

---

53. Technology Stack

Framework:

- Next.js (App Router)

Language:

- TypeScript

Styling:

- Tailwind CSS

Component Library:

- shadcn/ui

Animations:

- Framer Motion

Forms:

- React Hook Form
- Zod

State Management:

- Zustand

Server State:

- TanStack Query

Icons:

- Lucide React

Maps:

- Mapbox GL JS (or Google Maps if budget allows)

Charts:

- Recharts

Internationalization:

- next-intl

---

54. Folder Structure

```
app/
  (marketing)/
    page.tsx
    about/
    contact/
```

```
  (auth)/
    login/
    register/
    forgot-password/

  (dashboard)/
    dashboard/
    chat/
    opportunities/
    timeline/
    saved/
    profile/
    notifications/
    settings/

components/
  common/
  chat/
  cards/
  forms/
  layout/
  maps/
  opportunity/
  timeline/
  profile/
  admin/

hooks/

lib/

services/

store/

types/

utils/
```

---

55. Design System

Primary Colors

Primary:

- #2563EB

Success:

- #16A34A

Warning:

- #F59E0B

Danger:

- #DC2626

Background:

- #F8FAFC

Surface:

- #FFFFFF

Border:

- #E2E8F0

Text Primary:

- #0F172A

Text Secondary:

- #64748B

These colors communicate trust and professionalism.

---

56. Typography

Font:

- Inter

Scale:

Heading XL
48px

Heading L
36px

Heading M
28px

Heading S
24px

Body
16px

Small
14px

Caption
12px

---

57. Navigation Structure

Landing

↓

Authentication

↓

Dashboard

↓

AI Chat

↓

Recommendations

↓

Program Details

↓

Action Plan

↓

Timeline

↓

Saved Programs

↓

Profile

↓

Settings

Bottom navigation on mobile.

Sidebar navigation on desktop.

---

58. Landing Page

Purpose:

Introduce the platform and encourage users to start a conversation.

Sections:

Hero

Headline

"Tell us your situation. We'll help you discover opportunities."

CTA:

Start Conversation

Feature Highlights

Cards:

- Scholarships
- Healthcare
- Government Benefits
- NGO Support
- Agriculture
- Small Business
- Legal Aid
- Jobs

How It Works

1. Tell your story
2. AI understands your needs
3. Discover opportunities
4. Follow your action plan

Testimonials

Future implementation.

FAQ

Common questions.

Footer

Contact

Privacy

Terms

Support

---

59. Authentication

Pages:

Login

Register

Forgot Password

Email Verification

Profile Completion

Google Login

---

60. Dashboard

Purpose:

Provide a personalized overview.

Widgets:

Recent Conversation

Recommended Opportunities

Saved Opportunities

Upcoming Deadlines

Timeline

Notifications

Nearby Offices

Progress Tracker

Quick Actions

Quick Actions

Start Chat

Find Scholarships

Find Healthcare

Find Benefits

Find Jobs

Nearby Offices

---

61. AI Chat Page

The AI Chat page is the primary interaction surface.

Layout:

Sidebar

↓

Conversation History

↓

Main Chat Window

↓

Suggested Questions

↓

Input Box

Message types:

User

Assistant

System

Recommendation

Action Plan

Error

Capabilities

Voice Input

Markdown Rendering

Source References

Typing Indicator

Streaming Responses

Copy Response

Feedback Buttons

---

62. Recommendation Cards

Every recommendation is displayed as a card.

Card contains:

Title

Category

Organization

Eligibility Status

Confidence Score

Deadline

Benefits

Distance

Buttons

View Details

Save

Share

Apply

Example

Scholarship

Eligible

95%

Deadline

August 20

View Details

---

63. Opportunity Details Page

Displays complete information.

Sections:

Overview

Benefits

Eligibility

Required Documents

Application Process

Official Sources

Nearby Offices

Timeline

FAQs

Related Opportunities

---

64. Action Plan Page

Generated after recommendation.

Example:

Today

Collect National ID

↓

Tomorrow

Collect Income Certificate

↓

Visit Local Office

↓

Submit Application

↓

Track Status

Each task includes:

Priority

Estimated Time

Completion Status

Notes

---

65. Opportunity Timeline

A calendar-based experience.

Displays:

Deadlines

Appointments

Renewals

Reminders

Training

Applications

Users can switch between:

Day

Week

Month

Agenda

---

66. Saved Opportunities

Features:

Bookmark

Filter

Sort

Group

Track Status

Search

Categories:

Interested

Preparing

Applied

Completed

Rejected

---

67. Notifications

Types:

Application Reminder

Deadline Reminder

New Opportunity

Program Updated

Document Expiring

Recommendation Improved

Notification preferences:

Push

Email

Future SMS

---

68. Profile Page

Sections:

Personal Information

Education

Employment

Family

Income

Health (optional and user-controlled)

Interests

Saved Opportunities

Conversation History

Notification Settings

Privacy Settings

---

69. Settings

Language

Theme

Accessibility

Notification Preferences

Privacy

Delete Account

Export Data

---

70. Nearby Services

Interactive map.

Shows:

Government Offices

Hospitals

NGOs

Banks

Training Centers

Legal Aid

Agriculture Offices

Clicking a marker opens:

Name

Address

Hours

Phone

Navigation

---

71. Search Experience

Search supports:

Natural Language

Keywords

Voice

Filters

Suggested Searches

Recent Searches

Search filters:

Location

Category

Eligibility

Deadline

Organization

---

72. Accessibility

Support:

Keyboard Navigation

Screen Readers

High Contrast

Large Text

Reduced Motion

Voice Navigation

Color Blind Friendly Design

---

73. Loading States

Every async operation requires:

Skeleton Screens

Loading Spinner

Progress Indicator

Streaming Response Indicator

No blank pages.

---

74. Empty States

Instead of blank screens, provide guidance.

Example:

"No saved opportunities yet."

Button:

Explore Opportunities

---

75. Error States

Every error must be actionable.

Example:

Unable to retrieve scholarships.

Actions:

Retry

Refresh

Report Issue

---

76. Mobile Experience

Priority:

Mobile-first.

Features:

Bottom Navigation

Large Touch Targets

Voice-first Input

Responsive Cards

Swipe Gestures

Offline Drafts (future)

---

77. Admin Dashboard

Sections:

Programs

Organizations

Documents

Users

Analytics

AI Logs

Embeddings

Notifications

Rule Management

System Health

---

78. Analytics Dashboard

KPIs:

Daily Active Users

Monthly Active Users

Popular Programs

Search Volume

Recommendation Accuracy

Response Time

User Satisfaction

Application Conversions

---

79. Design Principles

- One primary action per screen.
- Maximum three levels of navigation.
- Clear visual hierarchy.
- Consistent spacing and typography.
- Accessible color contrast.
- Progressive disclosure for complex information.
- Every recommendation should lead to an actionable next step.

---

80. UX Principles

- Ask only for information that is necessary.
- Remember user-provided information to reduce repetition.
- Provide explanations before requesting sensitive data.
- Show confidence and sources for every recommendation.
- Make complex government processes feel simple.
- Prioritize speed, readability, and trust over decorative UI.

This completes **Part 5**. It specifies the entire frontend architecture, page structure, component behavior, navigation, accessibility, and design system.

The next section, **Part 6**, will be the **Development Blueprint**—covering repository structure, coding standards, branching strategy, implementation order, sprint planning, testing strategy, CI/CD, environment configuration, Docker setup, and the exact roadmap an AI coding assistant can follow to build the MVP efficiently.

AccessAI

Part 6 — Development Blueprint & Engineering Roadmap

> **Purpose:** This section defines how the engineering team (or an AI coding assistant such as Cursor, Claude Code, Codex, or Gemini CLI) should implement AccessAI

from an empty repository to a production-ready MVP. It establishes coding conventions, project structure, development workflow, testing strategy, deployment process, and implementation milestones.

---

81. Development Philosophy

The platform must be developed incrementally.

Every completed sprint should produce a deployable, testable application.

Follow these principles:

- Build vertical slices (feature by feature), not isolated layers.
- Keep the codebase modular from day one.
- Avoid premature optimization.
- Prioritize readability over cleverness.
- Keep business logic independent of UI.
- Every feature should be testable.
- Every API should be documented.
- Every database change should use migrations.

---

82. Repository Structure

The project should be organized as a monorepo.

access-ai/

```
├── apps/
│   ├── web/              # Next.js Frontend
│   ├── api/              # NestJS Backend
│   └── admin/            # Admin Panel (optional)
│
├── packages/
│   ├── ui/               # Shared UI Components
│   ├── types/            # Shared Types
│   ├── config/           # Shared Config
│   ├── prompts/          # AI Prompt Templates
│   ├── validation/       # Zod Schemas
```

```
│   ├── constants/
│   └── utils/
│
├── infrastructure/
│   ├── docker/
│   ├── nginx/
│   ├── terraform/
│   └── scripts/
│
├── docs/
│
├── .github/
│
└── README.md
```

---

83. Backend Folder Structure

src/

```
├── auth/
├── users/
├── ai/
├── chat/
├── opportunities/
├── eligibility/
├── search/
├── organizations/
├── documents/
├── embeddings/
├── notifications/
├── analytics/
├── admin/
├── common/
├── database/
├── config/
└── main.ts
```

Each module should contain:

module/

controller.ts

service.ts

repository.ts

dto/

entities/

interfaces/

validators/

tests/

---

84. Frontend Folder Structure

app/

components/

hooks/

providers/

services/

store/

types/

constants/

lib/

utils/

styles/

---

85. Coding Standards

General Rules:

- TypeScript strict mode enabled.
- No `any` unless absolutely necessary.
- Prefer interfaces for public contracts.
- Prefer composition over inheritance.
- Keep functions under ~50 lines where practical.
- Single Responsibility Principle.
- No duplicated business logic.
- Every exported function must include documentation comments.

Naming conventions:

Variables

camelCase

Functions

camelCase

Components

PascalCase

Classes

PascalCase

Database Tables

snake_case

API Routes

kebab-case

86. Git Workflow

Branch naming:

main

develop

feature/chat

feature/eligibility

feature/search

feature/opportunities

fix/auth

hotfix/login

release/v1

Commit message convention:

feat:

fix:

refactor:

test:

docs:

perf:

style:

chore:

Example:

feat(chat): implement streaming AI responses

fix(auth): resolve refresh token expiration bug

refactor(search): optimize hybrid retrieval logic

---

87. Environment Variables

Backend:

DATABASE_URL=

REDIS_URL=

JWT_SECRET=

JWT_REFRESH_SECRET=

OPENAI_API_KEY=

GOOGLE_MAPS_API_KEY=

SMTP_HOST=

SMTP_USER=

SMTP_PASSWORD=

S3_BUCKET=

S3_REGION=

S3_ACCESS_KEY=

S3_SECRET_KEY=

Frontend:

NEXT_PUBLIC_API_URL=

NEXT_PUBLIC_MAP_PROVIDER=

NEXT_PUBLIC_APP_NAME=

Never expose secrets to the client.

---

88. Database Migration Strategy

Use Prisma or TypeORM migrations consistently.

Every schema change must be versioned.

Migration naming:

001_create_users

002_create_opportunities

003_add_embeddings

004_notification_system

No manual production schema changes.

---

89. API Versioning

All APIs should be versioned.

Example:

/api/v1/auth/login

/api/v1/chat

/api/v1/opportunities

/api/v1/profile

Future changes should introduce `/v2` without breaking existing clients.

---

90. Prompt Management

AI prompts must **never** be hardcoded inside services.

Store prompts as version-controlled template files.

Example:

packages/prompts/

conversation.md

eligibility.md

recommendation.md

timeline.md

summarization.md

Advantages:

- Easier iteration.
- Version tracking.
- Prompt A/B testing.
- Non-developers can review prompts.

---

91. Development Order

The recommended implementation sequence is:

Phase 1

- Project setup
- Authentication
- User profile
- Database
- Landing page

Phase 2

- AI Chat
- Conversation memory
- Prompt orchestration
- Streaming responses

Phase 3

- Opportunity database
- Search
- Recommendation engine

Phase 4

- Eligibility engine
- Rule evaluation
- Action plans

Phase 5

- Maps
- Nearby services
- Notifications
- Timeline

Phase 6

- Admin dashboard
- Analytics
- Monitoring
- Optimization

---

92. Sprint Plan

Sprint 1

- Repository setup
- CI/CD
- Authentication
- Landing page
- User profile

Deliverable:

Working login system.

Sprint 2

- AI Chat
- Streaming
- Conversation history
- Prompt management

Deliverable:

Users can chat with AI.

Sprint 3

- Opportunity database
- Search

- Recommendation cards

Deliverable:

Users receive opportunity recommendations.

Sprint 4

- Eligibility engine
- Rule system
- Explanation engine

Deliverable:

Verified eligibility checking.

Sprint 5

- Timeline
- Notifications
- Saved opportunities
- Maps

Deliverable:

Complete MVP.

Sprint 6

- Admin panel
- Analytics
- Testing
- Performance optimization

Deliverable:

Production candidate.

---

93. Testing Strategy

Unit Tests

Test:

- Services
- Eligibility rules
- Utilities
- Validators

Integration Tests

Test:

- API endpoints
- Database queries
- Authentication
- Search
- Recommendation pipeline

End-to-End Tests

User journeys:

- Register
- Login
- Chat
- Receive recommendations
- Save opportunity
- View action plan
- Logout

AI Evaluation Tests

Measure:

- Intent accuracy
- Entity extraction accuracy
- Retrieval precision
- Recommendation quality
- Hallucination rate
- Response latency

---

94. CI/CD Pipeline

Pipeline:

Developer Push

↓

GitHub Actions

↓

Lint

↓

Type Check

↓

Unit Tests

↓

Integration Tests

↓

Build

↓

Docker Image

↓

Deploy

↓

Health Check

↓

Production

Deployment should stop automatically if any stage fails.

---

95. Docker Setup

Containers:

- Frontend
- Backend
- PostgreSQL
- Redis
- Nginx

Optional:

- Prometheus
- Grafana

Each service should have its own Dockerfile.

Use Docker Compose for local development.

---

96. Deployment Architecture

Production deployment:

Internet

↓

Cloudflare

↓

Nginx

↓

Next.js

↓

NestJS

↓

Redis

↓

PostgreSQL

↓

Object Storage

↓

OpenAI API

This architecture supports horizontal scaling in the future.

---

97. Documentation Requirements

Maintain documentation for:

- API (Swagger/OpenAPI)

- Database schema
- Environment setup
- Deployment guide
- Prompt templates
- Admin guide
- User guide
- Troubleshooting

Documentation should evolve with the codebase.

---

98. Definition of Done (DoD)

A feature is considered complete only if:

- Business logic implemented.
- UI completed.
- API documented.
- Validation added.
- Unit tests written.
- Integration tests passing.
- Error handling implemented.
- Loading states implemented.
- Accessibility checked.
- Responsive on mobile and desktop.
- Logging added where appropriate.
- Code reviewed.

---

99. MVP Acceptance Criteria

The MVP is considered ready when a user can:

1. Register and log in.
2. Complete their profile.
3. Describe a life situation.
4. Receive verified opportunity recommendations.
5. Understand eligibility reasons.
6. View required documents.
7. Generate an action plan.

8. Save opportunities.
9. Track deadlines.
10. Receive notifications.
11. Locate nearby service providers.
12. Access the application in both Bangla and English.

---

100. Long-Term Engineering Principles

As the platform evolves:

- Maintain backward API compatibility where possible.
- Keep AI prompts modular and version-controlled.
- Separate deterministic business logic from LLM reasoning.
- Monitor performance and AI quality continuously.
- Design every new feature to be observable, testable, and explainable.
- Favor maintainability and correctness over rapid feature expansion.

This completes **Part 6**. The remaining two sections—**Part 7 (Knowledge Base & Data Pipeline)** and **Part 8 (Deployment, Operations, Business Strategy, and Competition Readiness)**—will define how the platform ingests and maintains trusted knowledge, and how it is operated and positioned for real-world deployment and competitions.

Excellent. This is the final and most strategic section. Unlike the previous parts, this focuses on **how the platform becomes a real national product** and how it should be presented to judges, investors, government agencies, and future partners.

AccessAI

Part 8 — Deployment, Operations, Business Strategy & Competition Readiness

---

118. Production Deployment Strategy

The MVP should be deployable using a cloud-native architecture that balances simplicity, scalability, and cost.

Recommended Deployment Stack

- Frontend: Vercel (Next.js)
- Backend: Railway, Render, or Fly.io (NestJS)
- Database: Managed PostgreSQL (Neon, Supabase, or Railway)
- Cache: Upstash Redis
- Object Storage: Cloudflare R2 or AWS S3
- CDN & DNS: Cloudflare
- Monitoring: Grafana + Prometheus (or hosted alternatives)
- Error Tracking: Sentry
- Analytics: PostHog or Google Analytics
- AI Provider: OpenAI (GPT-5.5 or latest suitable model)

The architecture should allow migration to Kubernetes or a managed cloud platform (AWS/GCP/Azure) without major code changes.

---

119. Production Operations

Daily automated tasks should include:

- Refresh opportunity data.
- Re-index search.
- Regenerate embeddings for updated documents.
- Check for expired opportunities.
- Archive inactive records.
- Send scheduled notifications.
- Generate analytics reports.
- Run health checks.

Weekly tasks:

- Review AI feedback.
- Audit knowledge base updates.
- Verify official sources.
- Evaluate recommendation quality.
- Backup databases.

Monthly tasks:

- Security audit.
- Performance optimization.
- Dependency updates.
- Cost analysis.
- Disaster recovery testing.

---

120. Monitoring & Observability

The platform should expose operational metrics for both engineering and business stakeholders.

Technical metrics:

- API latency
- Error rate
- Database query time
- Cache hit ratio
- Queue backlog
- AI response time
- Embedding generation time

Business metrics:

- Daily Active Users
- Monthly Active Users
- Average session duration
- Opportunity views
- Saved opportunities
- Completed action plans
- Notification open rate
- Recommendation click-through rate

AI quality metrics:

- Intent classification accuracy
- Retrieval precision
- Hallucination rate
- Citation coverage
- User satisfaction score

---

121. Security & Privacy

AccessAI processes personal information and therefore should follow privacy-by-design principles.

Requirements:

- Encrypt data in transit (HTTPS/TLS).
- Encrypt sensitive data at rest.
- Hash passwords with Argon2.
- Implement role-based access control.
- Log administrative actions.
- Apply rate limiting to public APIs.
- Validate and sanitize all inputs.
- Restrict file uploads by MIME type and size.
- Protect against prompt injection and cross-site scripting.

Users should have control over:

- Exporting their data.
- Deleting their account.
- Managing notification preferences.
- Controlling profile visibility.

---

122. Sustainability & Maintainability

The platform should remain maintainable as it grows.

Engineering practices:

- Modular architecture.
- Automated testing.
- Infrastructure as Code.
- Versioned APIs.
- Versioned prompts.
- Versioned eligibility rules.
- Continuous documentation.

Knowledge management:

- Track document versions.
- Preserve update history.
- Record data source provenance.
- Flag outdated information automatically.

---

123. Business Model (Future)

The MVP is intended as a public-interest platform.

Potential long-term revenue streams (without compromising public access):

- Government service contracts.
- White-label deployments for ministries.
- Enterprise APIs for NGOs and development organizations.
- Analytics dashboards for policymakers.
- Premium integrations for educational institutions.
- API access for verified partners.

Core citizen-facing functionality should remain free.

---

124. National Impact

Potential benefits include:

- Increased awareness of public opportunities.
- Higher utilization of government programs.
- Reduced information inequality.
- Faster access to scholarships and financial aid.
- Improved healthcare navigation.
- Better coordination between citizens and public institutions.
- Increased transparency through explainable recommendations.

---

125. Roadmap Beyond MVP

Phase 1 — MVP

- AI conversation
- Eligibility engine
- Opportunity search
- Action plans
- Notifications

Phase 2

- OCR for uploaded documents
- Voice-first experience
- Offline support
- Regional language support
- Mobile application

Phase 3

- Government API integrations
- Application status tracking
- Secure document vault
- Personalized dashboards
- Family account support

Phase 4

- Predictive opportunity recommendations
- Cross-ministry service orchestration
- AI-powered form filling
- National-scale analytics
- Regional expansion to other countries

---

126. Demonstration Scenario (Competition Demo)

The live demonstration should be story-driven rather than feature-driven.

Scenario:

A university student from Rajshahi has recently graduated, has limited financial resources, and wants to pursue a master's degree abroad.

Demo flow:

1. User opens AccessAI.
2. User says: "I recently graduated and want to study abroad, but I don't have much money."
3. AI asks follow-up questions (CGPA, degree, preferred country).
4. Eligibility engine evaluates scholarship opportunities.
5. Retrieval engine fetches official scholarship data.
6. AI explains why each scholarship is recommended.
7. Action plan is generated with required documents and deadlines.
8. Nearby IELTS preparation centers are suggested.
9. User saves opportunities and receives deadline reminders.

This demonstrates the complete pipeline in a single coherent narrative.

---

127. Expected Judge Questions & Suggested Responses

Q: Why not just use ChatGPT?

Answer:

ChatGPT generates answers from general knowledge. AccessAI combines deterministic eligibility rules, verified government and NGO data, semantic retrieval, and explainable AI to provide personalized, evidence-backed recommendations rather than generic responses.

Q: How do you prevent hallucinations?

Answer:

Eligibility is determined by a rule engine, not the LLM. Responses are grounded using Retrieval-Augmented Generation (RAG) from verified sources, and every recommendation includes citations and confidence indicators.

Q: How will data remain up to date?

Answer:

A scheduled ingestion pipeline synchronizes official sources, regenerates embeddings for updated documents, and maintains version history. Administrative review ensures changes are verified before publication.

Q: Can this scale nationally?

Answer:

Yes. The system starts as a modular monolith for simplicity and can evolve into independently scalable services. Stateless APIs, managed databases, caching, and cloud deployment support nationwide growth.

---

128. Risks & Mitigation

Risk	Mitigation
Outdated information	Automated synchronization + manual verification
AI hallucinations	RAG + deterministic rule engine + citations
High API costs	Response caching, prompt optimization, selective retrieval
Rapid policy changes	Versioned knowledge base with scheduled updates
User mistrust	Explainable recommendations and transparent sources
Low digital literacy	Simple conversational UI, multilingual support, voice interaction

---

129. Key Performance Indicators (KPIs)

The success of AccessAI should be measured using both technical and societal metrics.

Technical:

- API response time < 3 seconds
- Recommendation precision > predefined target
- High citation coverage
- Low hallucination rate

User:

- Active users
- Returning users
- Saved opportunities
- Completed action plans
- User satisfaction

Impact:

- Number of opportunities discovered
- Number of successful applications initiated
- Number of citizens served
- Geographic coverage
- Diversity of supported programs

---

130. Final Vision Statement

AccessAI is more than an AI chatbot or a search engine. It is an intelligent public opportunity platform designed to transform how citizens interact with government, educational institutions, NGOs, and essential services.

By combining explainable artificial intelligence, verified knowledge retrieval, deterministic eligibility evaluation, and personalized action planning, AccessAI empowers citizens to discover opportunities they might never have known existed and to navigate complex public systems with confidence.

The long-term vision is to establish AccessAI as Bangladesh's trusted digital gateway for public opportunities—reducing information inequality, improving access to essential services, and strengthening the connection between citizens and the institutions created to serve them.

Overall Assessment

As a BIIN/APICTA competition project, this concept is **strong** because it addresses a real societal problem with a technically credible architecture. Its strengths include:

- Clear social impact and alignment with digital public service goals.
- Practical use of AI beyond a generic chatbot.
- Explainable, trustworthy recommendations through RAG and rule-based eligibility.
- Scalable architecture suitable for national deployment.
- Strong demo potential with realistic user journeys.

To maximize competitiveness, the implementation quality will matter as much as the idea. Judges are likely to focus on:

- Whether the knowledge base contains real, verified data.
- The quality and explainability of recommendations.
- A polished, intuitive user experience.
- Demonstrated end-to-end functionality rather than mockups.
- Evidence that the platform can be maintained and updated over time.

A working MVP with a polished demo, accurate recommendations, and a clear societal impact narrative would make this a compelling submission for Bangladesh ICT & Innovation Awards and a credible candidate for APICTA.